

AI 시대에 맞는 TDD 개발방법론

Agenda

- I. AI 시대에 TDD의 필요성
- II. AI 기반 TDD 실습 (Bowling KATA)
- III. Agentic Engineering은 Over-engineering일까?
- IV. 지속적인 발전을 위한 Agentic Engineering

교육 내용 및 학습 목표

- AI 시대에 TDD가 왜 다시 주목받고 있는지 살펴보고, AI 시대에 어울리는 TDD 방법론에 대해 알아본다.
- 끝으로 Agentic Engineering에 대한 방향성에 대해 살펴본다.

Chapter 01

AI 시대에 TDD의 필요성

SW 품질을 위한 Agentic Engineering

기존 TDD

TDD는 생산성보다 매우 높은 품질을 위한 방법론이다.

장점은 높은 SW 품질 확보이며,

단점으로는 테스트 코드 작성과 리팩토링에 드는 시간과 비용 때문에 느리고 비효율적이라는 인식이 존재했다.

AI 시대가 되면서 TDD가 더 편해졌다.

AI의 생산성의 도움을 받으면, 빠른 테스트코드 작성과 리팩토링이 가능하다.

이제 TDD는 더 이상 답답하고 부담스러운 방식이 아니다.

AI의 생산성이 결합되면서, 개발자는 높은 품질을 유지하면서도 실질적으로 활용 가능한 수준의 속도와 효율을 확보할 수 있게 되었다.

즉, AI 시대의 TDD는

“느리지만 좋은 방법”이 아니라,

“빠르면서도 품질을 보장하는 현실적인 전략”으로 재해석되고 있다.

1. 지시의 모호성 축소

- Text 기반으로 요구사항을 전달하면, 무엇을 만들어야 하는지 구체적이지 않을 수 있다.
- 반면 TDD는 Unit Test를 먼저 작성함으로써 AI에게 요구사항을 Code 기반으로 명확히 지시할 수 있다.
- 사람도 Text 지시에 대해 모호성을 겪지만, AI는 이를 더 크게 확대해석한다.
- 테스트는 “무엇을 만들어야 하는 지”를 명확하게 고정해준다.
- Text가 아닌 코드 기반 명세이기 때문에 해석 오차가 줄어든다.
- 즉, Text 지시가 아닌 Code로 지시하면, 잘못된 방향으로 구현될 확률을 낮춘다.

2. 안전장치 역할 (가장 중요)

- AI가 생성하는 코드는 빠르지만, 항상 신뢰할 수 있는 것은 아니다.
- TDD는 다음과 같은 구조를 만든다.
 - 먼저 테스트를 정의한다 (기대 결과 고정)
 - 그 다음 AI에게 구현을 맡긴다.
 - 여기서 Unit Test는 가드레일(안전장치) 역할을 한다.
 - 잘못된 구현은 Test Fail로 즉시 드러난다
 - 예상 범위를 벗어나는 코드가 만들어지기 어렵다.

3. 검토(리뷰)가 쉬워진다

- Test Code를 고민하면서 만들었다면, Green Code가 만들어지기 전 어떤 코드가 만들어질지 예상이 된다.
- 예측 범위 내에서 코드가 만들어지기에 AI가 만든 코드의 리뷰 속도가 굉장히 빠르다.

Chapter 02

AI 기반 TDD 실습 (Bowling KATA)

SW 품질을 위한 Agentic Engineering

AI 시대에 맞는 TDD

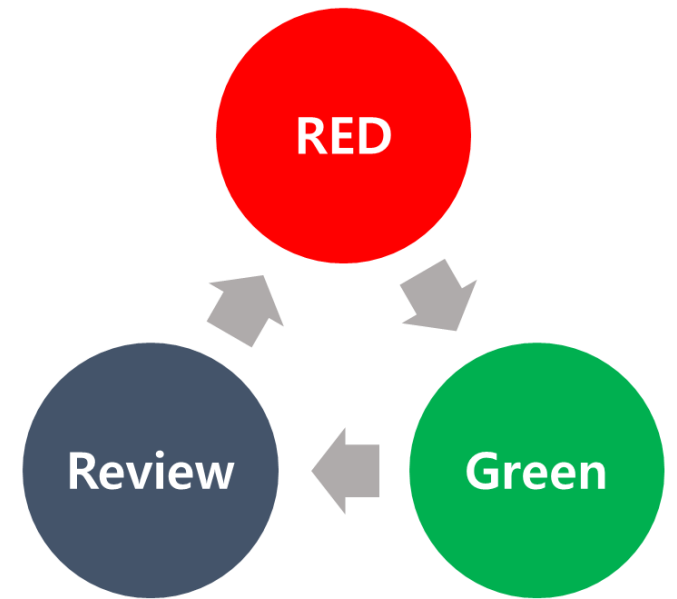
AI 시대에는 기존 방식에 더해, AI를 효과적으로 활용할 수 있는 Customize TDD가 필요하다. 이번 실습에서는 두 가지 방법의 AI + TDD를 단계적으로 경험해본다.

1. Original TDD 방식에 AI를 활용

- 기존 TDD 원칙을 유지한 상태에서 AI를 보조 도구로 활용한다.
- Bowling KATA를 통해 AI + TDD 사이클을 경험한다. (Superpower Skills 사용)

2. Agentic Engineering을 적용한 TDD 방식

- AE 원칙을 적용한 Red-Green-Review Cycle을 이해하고 경험한다.
- Bowling KATA를 Agentic Engineering 방식으로 재구성하여 실습한다.



Bowling kata 소개

Bowling 점수 산출 코드 작성 TDD

볼링 kata는 클린코드 저자로 유명한 Robert C. Martin 이 만든 TDD 연습용 kata이다.

실제 볼링 게임을 만드는 것이 아니라,

볼링 점수를 계산하는 로직과 설계를 TDD Cycle 로 연습해본다.

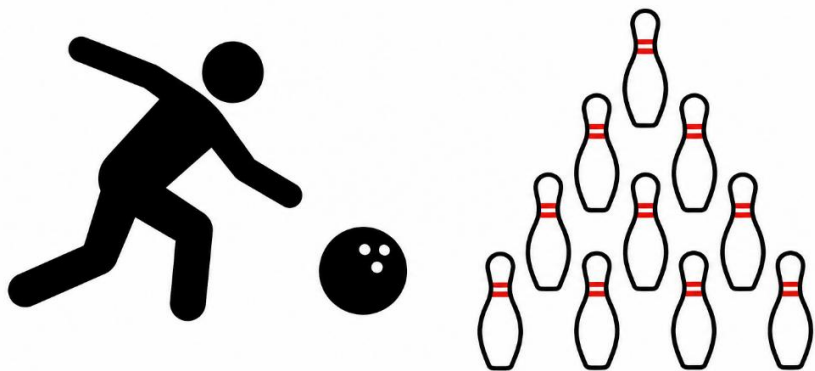
→ Coding Agent를 사용해서 TDD Cycle 을 돌려본다

Bowling 규칙

볼링 경기는 10 개의 볼링 핀을 넘어뜨리는 게임이다.

총 10 라운드가 진행되며, 각 라운드별로 최대 2번의 볼링공을 굴릴 기회가 주어진다.

다음 라운드에 볼링 핀은 다시 10개로 초기화된다.



각 라운드별로 **최대** 2번
굴릴 기회가 있다.

10 라운드는 예외로
최대 3번까지 굴릴 기회가 주어진다

1	2	3	4	5	6	7	8	9	10

볼링 점수판

기본 점수 규칙

한 라운드에서 얻을 수 있는 볼링의 점수는 공을 굴러 넘어뜨린 핀의 개수로 결정된다.
단, Spare 와 Strike 는 보너스 점수가 있다.

Round	
3	2

첫번째에는 3개, 두번째에는 2개를 쓰러뜨렸다.
→ 해당 라운드에서 얻은 점수 = $3 + 2$

Round	
2	/

첫번째에는 2개, 두번째에는 8개를 쓰러뜨려
한 라운드에 10개 모두 쓰러뜨렸다
→ 이를 Spare 라고 부른다.
→ 해당 라운드에서 얻은 점수 = $2 + 8 + \text{보너스}$

Round	
X	

첫번째 시도에 10개를 모두 쓰러뜨렸다.
→ 이를 Strike 라고 부른다.
→ 해당 라운드에서 얻은 점수 = $10 + \text{보너스}$

Spare 와 Strike 의 보너스 점수 규칙

한 라운드에 10개의 볼링 핀을 넘어뜨리면 Spare 또는 Strike 로 판정된다.

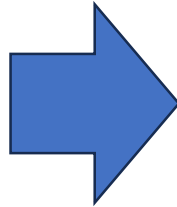
Spare 인 경우 **다음 1 번의 투구**에서 쓰러뜨린 핀 수를 보너스로 하고,

Strike 인 경우 **다음 2 번의 투구**에서 쓰러뜨린 핀 수를 보너스로 합산한다.

1Round		2Round	
2	/	3	5

1 Round 에서 얻는 점수
 $= 2 + 8 + 3$

1Round		2Round	
X		3	5



1 Round 에서 얻는 점수
 $= 10 + 3 + 5$

1Round		2Round		3Round	
X		X		X	

Q. 1,2,3 Round 에서 모두 Strike 인 경우
1 Round 에서 얻는 점수는?

마지막 10 Round 보너스

마지막 10 Round 는 Spare, Strike 인 경우 보너스 점수용 기회 포함 **3번 공을 굴릴 수 있다.**
단, 보너스 점수용 기회는 Strike이나 Spare이더라도 추가 기회는 없다.

10Round		
2	/	보너스

Spare 이므로 보너스 점수용 기회를 **1번** 준다.

10Round		
X	보너스	보너스

Strike 이므로 보너스 점수용 기회를 **2번** 준다.

Bowling 점수 예시

다음 볼링 점수판 예시를 보고 점수 규칙을 잘 이해했는지 확인해본다.

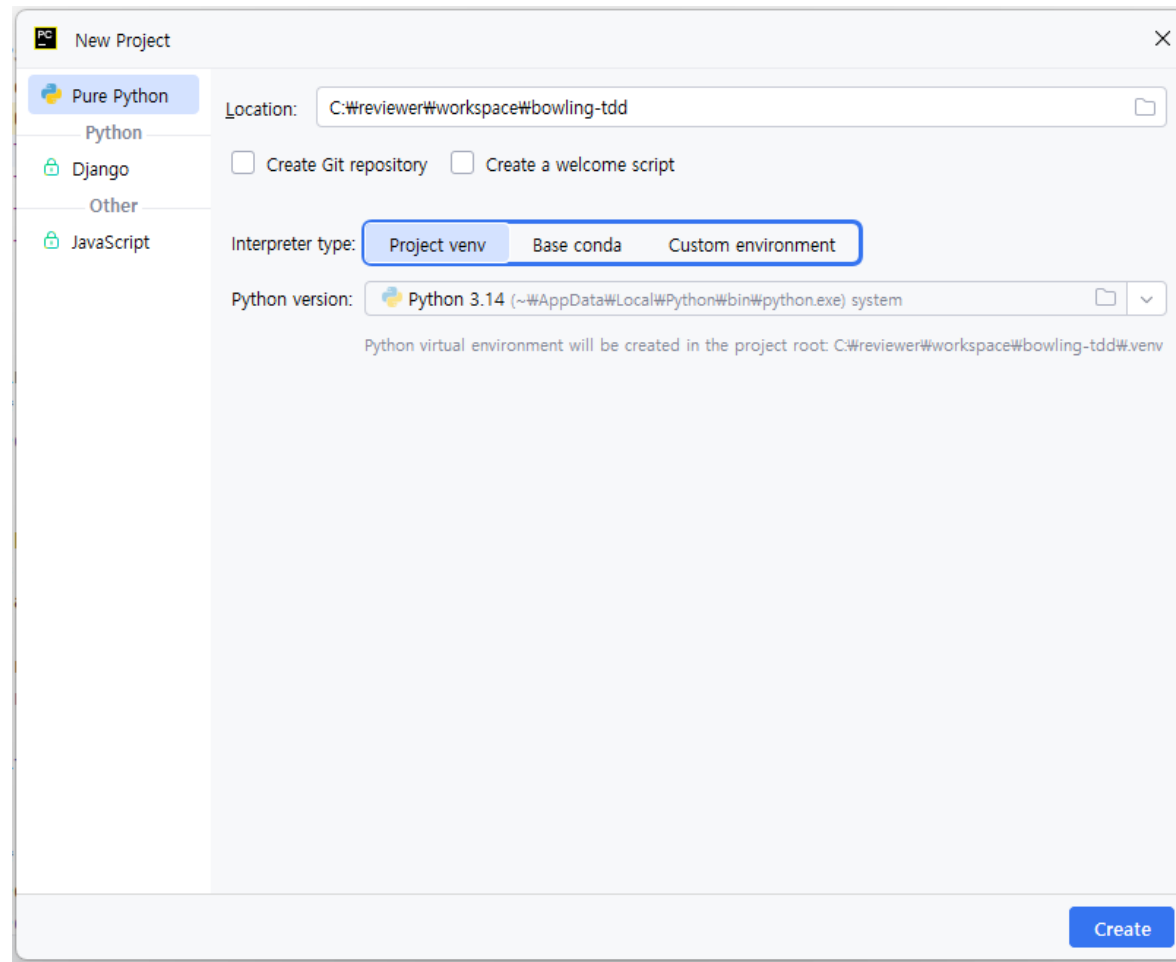
12	1	2	3	4	5	6	7	8	9	10	TTL
David	8 /	9 -	9 /	X	7 1	X	6 3	X	6 3	X 8 -	148
	19	28	48	66	74	93	102	121	130	148	
Smith	X	9 /	7 -	9 /	X	X	8 /	X	9 /	9 / ⑦	188
	20	37	44	64	92	112	132	152	171	188	
Green	4 -	5 -	8 -	X	1 8	1 -	6 /	8 -	9 -	7 1	89
	4	9	17	36	45	46	64	72	81	89	
Lazzari	9 /	5 4	3 /	7 /	X	6 -	6 -	⑧ 1	5 4		107
	15	24	41	61	77	83	89	98	107		
Hdcp	2/3	58	98	150	227	288	334	387	443	489	532

1 번째 쓰러뜨린 개수	2 번째 쓰러뜨린 개수
1 ~ n 라운드 누적점수	

Python 프로젝트 생성

Python 프로젝트 생성

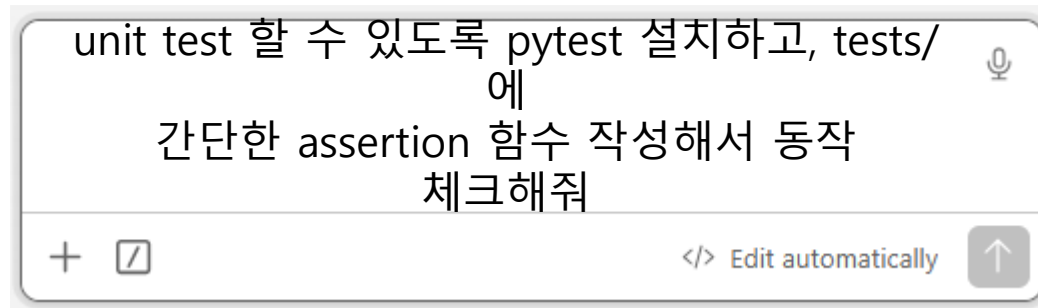
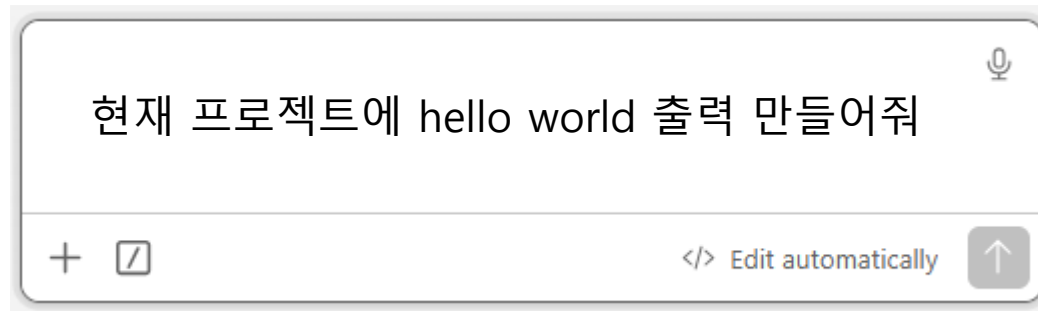
Project Name : bowling-tdd



프로젝트 세팅

agent 에게 기본 실습 세팅 요청하고, 정상적으로 동작되는지 체크까지 진행한다.

*** 동작 체크용 파일은 임시 파일이므로 지우도록 한다**



CLAUDE.md 생성

/init skill을 사용하여 CLAUDE.md 문서 생성 후

Bowling kata 내용 기입하기

기입할 내용: <https://gist.github.com/jeonghwan-seo/662e65f320fe0f0f3ce654ed464e89a8>

/init 빌드 및 실행 방법 일반적으로 정리

+ [icon] </> Edit automatically [up arrow]

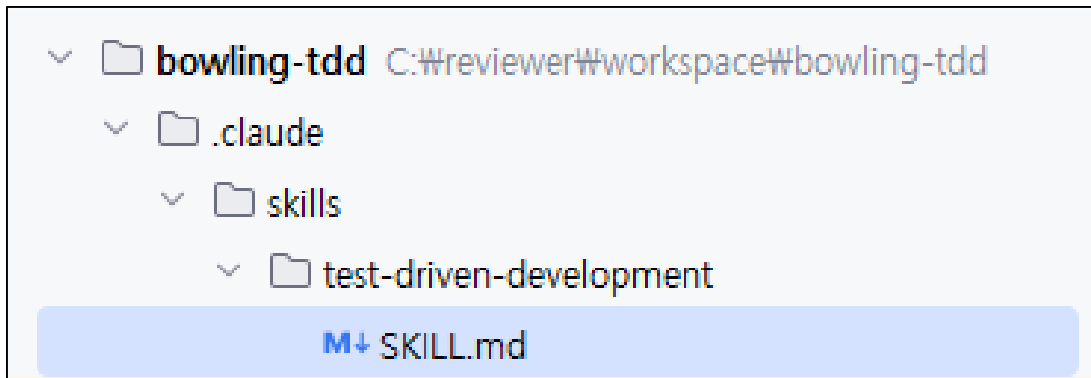
/init (+bowling kata 내용 기입)

+ [icon] </> Edit automatically [up arrow]

Skill 세팅하기

이번 실습은 superpowers 의 TDD Skill 을 가져와서 사용해본다.

- superpowers TDD skill : <https://github.com/obra/superpowers/tree/main/skills/test-driven-development>
- 언어별 번역본 : <https://gist.github.com/jeonghwan-seo/fd1a2cd6a2d3cc00b8a3d5a68a21cc2f>



다음과 같은 구조로 Skill 을 등록한다.

TDD Cycle 검토

기본적으로 Skill 에 명시된 대로 TDD 가 진행되는지 확인한다.

RED 단계에서 검토해야 할 부분 :

- ✓ 테스트는 구현에 대한 계획과 목표점이 된다.
- ✓ 테스트 작성을 제일 먼저 진행했는지
- ✓ 요구사항에 맞는 테스트 코드를 만들고 테스트 수행을 했는지

GREEN 단계에서 검토해야 할 부분:

- ✓ RED 단계의 테스트가 요구를 벗어난 불필요한 코드를 생성하진 않았는지 검토
- ✓ 테스트를 수행하고 PASS 되는지 확인

스킬 명시적 사용

스킬은 따로 언급하지 않아도 자연스럽게 발동될 수 있다.

하지만, 확실하게 사용하고 싶으면 명시적으로 스킬을 언급하도록 한다.

Bowling kata 구현해줘

+ [icon] </> Edit automatically [up arrow]

```
> 볼링 카타 구현해줘  
• 볼링 카타를 TDD 원칙대로 구현하겠습니다. .claude/skills/test-driven-development 가이드에 따라 RED → GREEN을 사이클별로 직접 보면서 진행합니다.
```

TDD 로 Bowling kata 구현해줘
test-driven-development 스킬 따라서 Bowling kata 구현해줘.
/test-driven-development Bowling kata 요구사항 파악하고 구현해줘

+ [icon] </> Edit automatically [up arrow]

```
> tdd 로 볼링 카타 구현해줘  
● Skill(test-driven-development)  
  ↳ Successfully loaded skill
```



명시적인 사용

요구사항 파악 및 TDD 개발 진입

요구사항이 정리 되어있는 문서를 잘 읽고 이해했는지 파악한다.

- 먼저 현재 프로젝트 구조를 파악하겠습니다.

Searched for 3 patterns, read 2 files (ctrl+o to expand)

- 프로젝트 구조를 파악했습니다. Java 소스가 없는 깨끗한 스캐폴드입니다. CLAUDE.md의 볼링 요구사항을 바탕으로 TDD로 구현하겠습니다.

요구사항 정리:

- Game 클래스: `void roll(int pins), int score()`
- 스페어: 다음 1구 보너스
- 스트라이크: 다음 2구 보너스
- 10번 프레임: 스트라이크/스페어 시 추가 투구

TDD 사이클: 실패하는 테스트 → GREEN → 다음 테스트 순서로 진행합니다.

RED 1 – 거터 게임 (전부 0)

가장 단순한 기능에 대한 테스트를 추가하고, 테스트 실패를 확인한다.

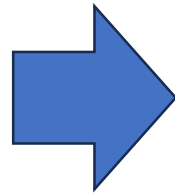
- ✓ 스트라이크, 스페어 등의 여러 요구사항을 배제한 테스트부터 시작
- ✓ 코드 뼈대 형성 가능

RED 1 – 거터 게임 (전부 0)

먼저 테스트 파일을 작성합니다.

● RED 1 검증 – 실패 확인

```
● Write(tests\test_game.py)
  L Wrote 10 lines to tests\test_game.py
  1 import pytest
  2 from game import Game
  3
  4
  5 def test_gutter_game():
  6     """All rolls are gutter (0 pins)"""
  7     game = Game()
  8     for _ in range(20):
  9         game.roll(0)
 10     assert game.score() == 0
```



● RED 검증 – 테스트가 실패하는 것을 확인합니다.

```
● Bash(python -m pytest tests/test_game.py::test_gutter_game -v)
  L Error: Exit code 4
  ===== test session starts =====
  platform win32 -- Python 3.10.8, pytest-7.2.1, pluggy-1.0.0 -
  C:\Users\hwany\AppData\Local\Programs\Python\Python310\python
  cachedir: .pytest_cache
  rootdir: C:\reviewer\workspace\bowling-tdd
  plugins: anyio-4.11.0
```

GREEN 1 – 최소 구현으로 통과

RED 1의 테스트를 통과시킨다

GREEN 1 – 최소 구현으로 통과

- Write(src\main\java\org\example\bowling\Game.java)
 - └ Wrote 19 lines to src\main\java\org\example\bowling\Game.java

```
● Write(C:\reviewer\workspace\bowling-tdd\game.py)
└ Wrote 9 lines to C:\reviewer\workspace\bowling-t
  1 class Game:
  2     def __init__(self):
  3         self._rolls = []
  4
  5     def roll(self, pins):
  6         self._rolls.append(pins)
  7
  8     def score(self):
  9         return 0
```

```
● PowerShell(cd "C:\reviewer\workspace\bowling-tdd"; .\.venv\Scripts
└ ===== test session starts =====
platform win32 -- Python 3.14.4, pytest-9.0.3, pluggy-1.6.0 --
C:\reviewer\workspace\bowling-tdd\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\reviewer\workspace\bowling-tdd
collecting ... collected 1 item

tests/test_game.py::test_gutter_game PASSED

===== 1 passed in 0.02s =====
```

이전 RED 단계에서 작성한 테스트를 실행
→ PASS

RED 2 – 전부 1개의 볼링핀

점수 누적에 대한 테스트 추가

```
---
Cycle 2: All Ones (RED)
● Update(C:\reviewer\workspace\bowling-tdd\tests\test_game.py)
  L Added 7 lines
    6     for _ in range(20):
    7         game.roll(0)
    8     assert game.score() == 0
    9 +
   10 +
   11 +def test_all_ones():
   12 +    game = Game()
   13 +    for _ in range(20):
   14 +        game.roll(1)
   15 +    assert game.score() == 20
```

```
Error: Exit code 1
===== test session =====
platform win32 -- Python 3.14.4, pytest-9.
C:\reviewer\workspace\bowling-tdd\.venv\Sc
cachedir: .pytest_cache
rootdir: C:\reviewer\workspace\bowling-tdd
collecting ... collected 1 item

tests/test_game.py::test_all_ones FAILED
```

GREEN 2 – 전부 1개의 볼링핀

점수 누적에 대한 구현

- RED 확인. GREEN – `score()`가 실제 합산을 하도록 수정합니다.

- Update(game.py)

- └ Added 1 line, removed 1 line

- 6 `self._rolls.append(pins)`

- 7

- 8 `def score(self):`

- 9 - `return 0`

- 9 + `return sum(self._rolls)`

```
===== test session starts =====
platform win32 -- Python 3.14.4, pytest-9.0.3, pluggy-1.6.0 --
C:\reviewer\workspace\bowling-tdd\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\reviewer\workspace\bowling-tdd
collecting ... collected 2 items

tests/test_game.py::test_gutter_game PASSED
tests/test_game.py::test_all_ones PASSED

===== 2 passed in 0.02s =====
```

RED 3 – 스페어 테스트 추가

스페어 점수에 대한 테스트 추가

```
---
Cycle 3: Spare (RED)
● Update(C:\reviewer\workspace\bowling-tdd\tests\test_game.py)
  L Added 10 lines
    13     for _ in range(20):
    14         game.roll(1)
    15     assert game.score() == 20
    16 +
    17 +
    18 +def test_one_spare():
    19 +    game = Game()
    20 +    game.roll(5)
    21 +    game.roll(5) # spare
    22 +    game.roll(3) # bonus counts once
    23 +    for _ in range(17):
    24 +        game.roll(0)
    25 +    assert game.score() == 16
```

spare 의 보너스 규칙에 대한
테스트를 생성했는지 검토

```
Error: Exit code 1
```

```
===== test session starts =====
platform win32 -- Python 3.14.4, pytest-9.0.3, pluggy-1.6.0 --
C:\reviewer\workspace\bowling-tdd\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\reviewer\workspace\bowling-tdd
collecting ... collected 1 item
```

```
tests/test_game.py::test_one_spare FAILED
```

```
===== FAILURES =====
```

- 예상대로 실패합니다 (보너스 미적용 → 13점, 기대값 16점).

스페어에 대한 기능 추가 전
→ FAIL 확인

GREEN 3 – 스페어 보너스 구현

Spare 점수에 대한 구현 추가

GREEN 3 – 스페어 보너스 구현

- Update(src\main\java\org\example\bowling\Game.java)
 - └ Added 13 lines, removed 2 lines

```
11
12     public int score() {
13         int score = 0;
14 -     for (int i = 0; i < 20; i++) {
15 -         score += rolls[i];
14 +     int rollIndex = 0;
15 +     for (int frame = 0; frame < 10; frame++) {
16 +         if (isSpare(rollIndex)) {
17 +             score += 10 + rolls[rollIndex + 2];
18 +             rollIndex += 2;
19 +         } else {
20 +             score += rolls[rollIndex] + rolls[rollIndex + 1];
21 +             rollIndex += 2;
22 +         }
23     }
24     return score;
25 }
26 +
27 + private boolean isSpare(int rollIndex) {
28 +     return rolls[rollIndex] + rolls[rollIndex + 1] == 10;
29 + }
30 }
31
```

RED 단계의 Spare 점수에 대한 테스트 실행
→ PASS 가 되어야 함

RED 4 – 스트라이크 테스트 추가

스트라이크 점수에 대한 테스트 추가

```
---  
Cycle 4: Strike (RED)
```

```
def test_one_strike():  
    game = Game()  
    game.roll(10) # strike  
    game.roll(3)  
    game.roll(4) # bonus counts twice  
    for _ in range(16):  
        game.roll(0)  
    assert game.score() == 24
```

스트라이크 보너스 점수에 대한
테스트를 생성했는지 검토

스트라이크에 대한 기능 추가 전
→ FAIL 확인

GREEN 4 – 스트라이크 보너스 구현

스트라이크 기능에 대한 테스트가 통과할 수 있도록 구현

```
def score(self):
    total = 0
    roll_index = 0
    for frame in range(10):
        if self._rolls[roll_index] == 10: # strike
            total += 10 + self._rolls[roll_index + 1] + self._rolls[roll_index + 2]
            roll_index += 1
        elif self._rolls[roll_index] + self._rolls[roll_index + 1] == 10: # spare
            total += 10 + self._rolls[roll_index + 2]
            roll_index += 2
        else:
            total += self._rolls[roll_index] + self._rolls[roll_index + 1]
            roll_index += 2
    return total
```

RED/GREEN 5 – 퍼펙트 게임 테스트(모두 스트라이크)

마지막 10라운드에서의 규칙도 적용되도록 테스트와 구현을 한다.

```
---  
Cycle 5: Perfect Game (12 strikes)  
  
RED
```

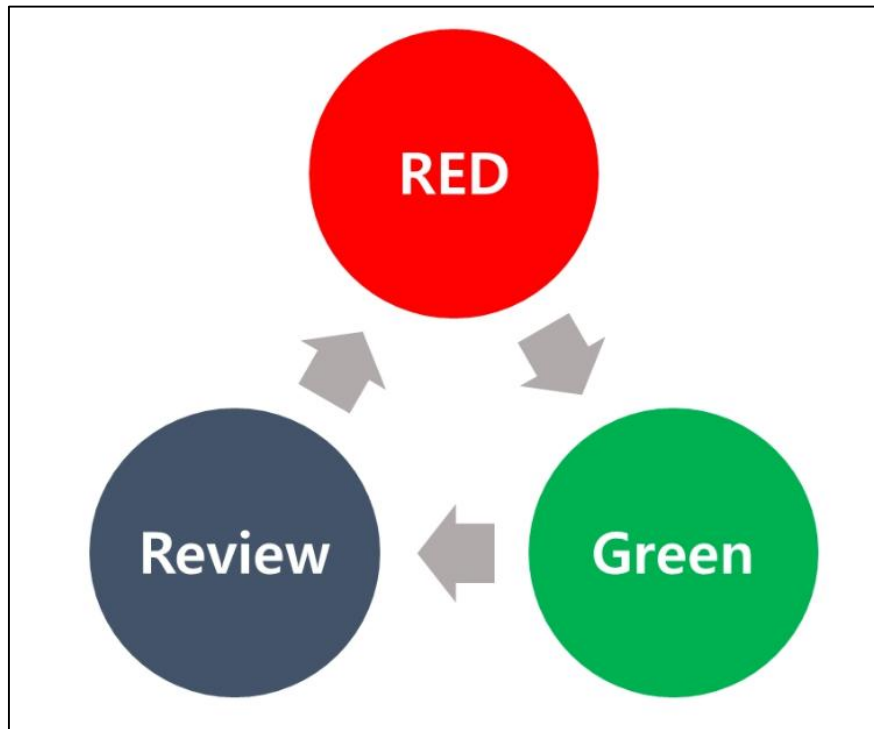
```
def test_perfect_game():  
    game = Game()  
    for _ in range(12):  
        game.roll(10)  
    assert game.score() == 300
```

```
● PowerShell(cd "C:\reviewer\workspace\bowling-tdd"; .\venv\Scripts\python.exe  
    tests/test_game.py::test_perfect_game -v)  
  L ===== test session starts =====  
platform win32 -- Python 3.14.4, pytest-9.0.3, pluggy-1.6.0 --  
C:\reviewer\workspace\bowling-tdd\.venv\Scripts\python.exe  
cachedir: .pytest_cache  
rootdir: C:\reviewer\workspace\bowling-tdd  
collecting ... collected 1 item  
  
tests/test_game.py::test_perfect_game PASSED  
  
===== 1 passed in 0.02s =====  
  
● 퍼펙트 게임도 즉시 PASSED - 프레임 순회 로직이 이미 올바르게 동작합니다.  
전체 테스트 GREEN 확인
```

새로운 테스트, but 기존 구현으로 통과
→ 테스트가 이전 구현으로 우연히 통과인지 체크
→ 이전 구현이 너무 과도하지 않았는지 체크

Agentic Engineering TDD 리뷰

human-in-the-loop 를 강조하여 Agent 를 사용한 TDD 를 진행해본다.



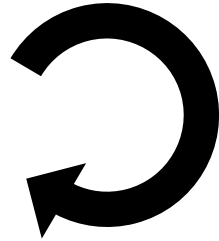
1. RED
 - 목표 정확히 설정
 - Plan.md 생성 요청 후 검토
2. Green
 - RED 에서 생성된 goal 을 달성하도록 구현
 - 테스트를 실제로 통과하는지 체크
3. Review
 - Green 의 코드를 리뷰
 - Plan 외의 사항이 구현되었는지
 - 리팩토링이 필요하다면 요청하기

프로젝트 수행시 커밋할 부분

agentic engineering 수행을 체크하기 위한 용도로,
각 단계별 필요한 커밋들을 생성한다.



RED 단계에서 생성된
Plan.md Commit



GREEN 단계의 코드를 검토 후 Commit



프로젝트 제출방법 - 프로젝트

GitHub 에 Remote Repository를 생성

Repository 이름

- repo 이름 : **bowling-사번**
- Description : 소속 & 이름
- 접근 권한 : Public

Create a new repository

A repository contains all project files, including the revision history.

Owner

Repository name *

 reviewer-01 ▾

 /

bowling-사번

Great repository names are short and memorable. Need inspiration? How about [redesigned-journey?](#)

Description (optional)

소속 & 이름

☒  Public

Any logged in user can see this repository. You choose who can commit.

☐  Internal

Samsung SDS [enterprise members](#) can see this repository. You choose who can commit.

☐  Private

You choose who can see and commit to this repository.

Skip this step if you're importing an existing repository.

☐ Initialize this repository with a README

This will let you immediately clone the repository to your computer.

Add .gitignore: [None](#) ▾

Create repository

Skill Update하기

human-in-the-loop 를 강조하여 TDD 를 진행하기 위해,
필요한 내용들을 적은 후 기존 tdd 스킬을 업데이트한다.

```
> @skill-update-plan.md 를 파악해서 local project 의 /test-driven-development skill 을 업데이트  
해 줘  
L Read skill-update-plan.md (32 lines)
```

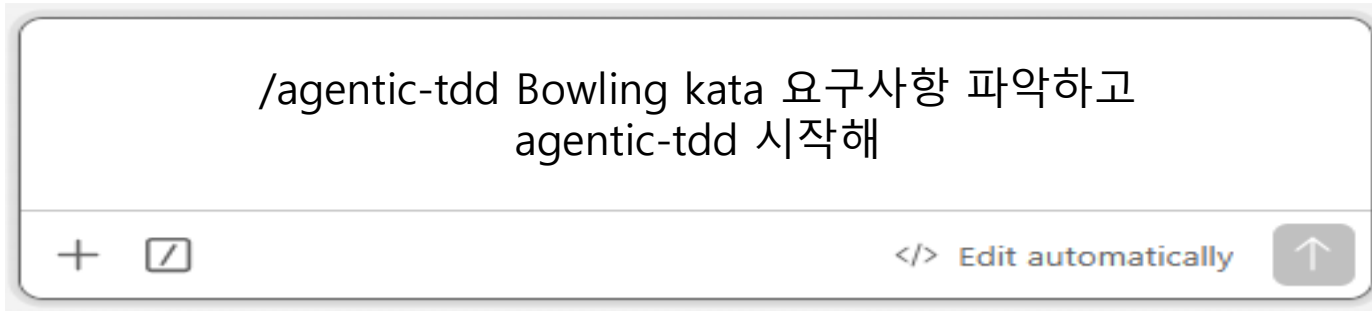
agentic engineering 을 TDD 에서 실천할 수 있도록
문서를 작성해놓는다.

새롭게 작성된 Skill 가동

update 된 skill 을 사용해서 Agentic Engineering 에 맞는 TDD 를 진행한다.

- 한 사이클을 수행하면서 Agentic Engineering 을 잘 지키는지 Skill 검토가 필요하다.
- Agentic Engineering 을 잘 수행하지 않으면 Skill을 수정한다.

```
/agentic-tdd Bowling kata 요구사항 파악하고  
agentic-tdd 시작해
```



RED 1 – Plan.md

Agent 가 작성한 Plan.md 를 검토한다.

```
● 요구사항을 파악했습니다. 빈 프로젝트(아직 game.py, tests/test_game.py 없음)에서 시작하며, ag
따르겠습니다.

첫 사이클의 Plan.md를 작성합니다.

● Write(C:\reviewer\workspace\bowling-tdd\Plan.md)
  L Wrote 52 lines to C:\reviewer\workspace\bowling-tdd\Plan.md
    1 # 사이클 1 - 거터 게임 (Gutter Game)
    2
    3 ## 목표 동작 (Behavior)
    4 20번의 롤이 모두 0핀일 때, 'Game.score()'가 0을 반환한다.
    5
    6 ## 테스트 케이스
    7 - 테스트 이름: 'test_gutter_game'
    8 - 입력 / 셋업: 새로운 'Game()' 인스턴스 생성 후 'roll(0)'을 20회 호출
    9 - 기대 결과 (assertion): 'game.score() == 0'
   10 - 왜 이 케이스가 가치 있는가?
   11   - 가장 단순한 게임 시나리오로, 'Game'의 기본 골격('__init__', 'roll', 'score')을 강
   12   - 스페어/스트라이크가 없는 "정상 프레임"의 합산 로직을 가장 단순한 형태로 검증한다.
   13   - 이후 사이클(올원 → 스페어 → 스트라이크 → 퍼펙트 → 10프레임)의 토대가 된다.
   14
   15 ## 다루지 않는 것 (Out of Scope)
   16 - 0이 아닌 핀 수의 처리 (다음 사이클: "All Ones")
   17 - 스페어/스트라이크 보너스 로직
   18 - 10프레임 특수 규칙
   19 - 입력 검증 (CLAUDE.md에 명시적으로 out of scope)
   20 - 중간 프레임 점수 조회
   21
   22 ## 예상 실패 메시지
   23 'game.py' 파일과 'Game' 클래스가 아직 존재하지 않으므로:
```

```
Plan.md
1 # 사이클 1 - 거터 게임 (Gutter Game)
2
3 ## 목표 동작 (Behavior)
4 20번의 롤이 모두 0핀일 때, 'Game.score()'가 0을 반환한다.
5
6 ## 테스트 케이스
7 - 테스트 이름: 'test_gutter_game'
8 - 입력 / 셋업: 새로운 'Game()' 인스턴스 생성 후 'roll(0)'을 20회 호출
9 - 기대 결과 (assertion): 'game.score() == 0'
10 - 왜 이 케이스가 가치 있는가?
11   - 가장 단순한 게임 시나리오로, 'Game'의 기본 골격('__init__', 'roll', 'score')을 강제한다.
12   - 스페어/스트라이크가 없는 "정상 프레임"의 합산 로직을 가장 단순한 형태로 검증한다.
13   - 이후 사이클(올원 → 스페어 → 스트라이크 → 퍼펙트 → 10프레임)의 토대가 된다.
14
15 ## 다루지 않는 것 (Out of Scope)
16 - 0이 아닌 핀 수의 처리 (다음 사이클: "All Ones")
17 - 스페어/스트라이크 보너스 로직
18 - 10프레임 특수 규칙
19 - 입력 검증 (CLAUDE.md에 명시적으로 out of scope)
20 - 중간 프레임 점수 조회
21
22 ## 예상 실패 메시지
23 'game.py' 파일과 'Game' 클래스가 아직 존재하지 않으므로:
24
25 ...
26 ModuleNotFoundError: No module named 'game'
27
28 (또는 'tests/test_game.py' 파일이 새로 작성되어야 하므로 임포트 단계에서 실패)
29
30 ## 파일 구조 (이 사이클에서 만들 것)
31 - 'tests/__init__.py' (필요 시 빈 파일)
32 - 'tests/test_game.py' - 'test_gutter_game' 1개 테스트
33 - (RED 후) 'game.py' - 'Game' 클래스 최소 구현
34
35 ## GREEN 단계 예상 코드 스케치 (참고용, 실행은 RED 후 작성)
36 ```python
37 class Game:
38     def __init__(self):
39         self._rolls = []
40
41     def roll(self, pins):
```

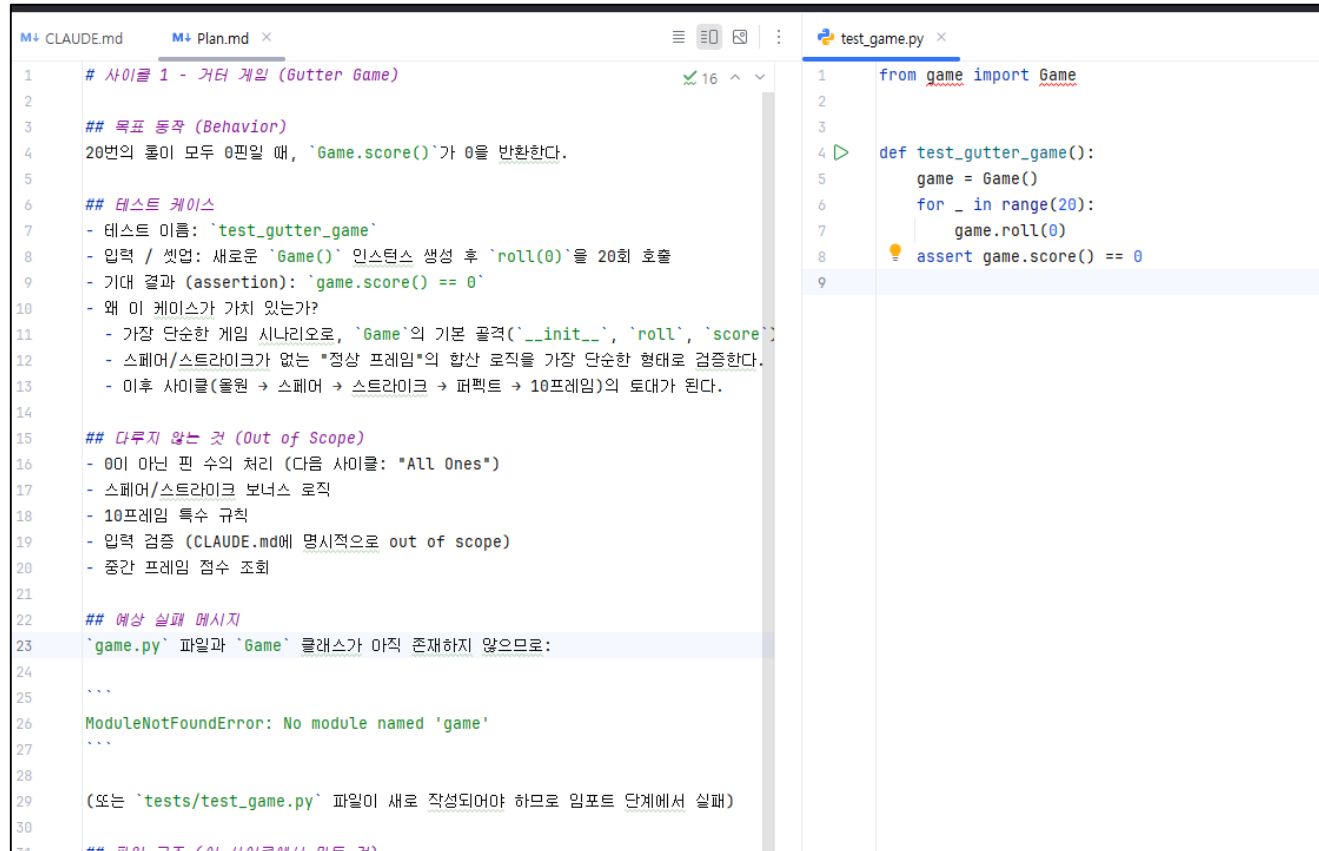
AI의 결과는 예시로
실제 실행 시 차이가 발생할 수 있습니다.

RED 1 – Test Code 변환 후 검토

Plan.md 가 마음에 들었다면 다음 step 을 진행시킨다.

Plan.md → Test Code (plan) 으로 변환작업을 수행

사람은 Plan 에 맞게 Test Code 를 만들었는지 확인한다.



```
M4 CLAUDE.md M4 Plan.md x test_game.py x
1 # 사이클 1 - 거터 게임 (Gutter Game)
2
3 ## 목표 동작 (Behavior)
4 20번의 롤이 모두 0핀일 때, 'Game.score()'가 0을 반환한다.
5
6 ## 테스트 케이스
7 - 테스트 이름: 'test_gutter_game'
8 - 입력 / 셋업: 새로운 'Game()' 인스턴스 생성 후 'roll(0)'을 20회 호출
9 - 기대 결과 (assertion): 'game.score() == 0'
10 - 왜 이 케이스가 가치 있는가?
11   - 가장 단순한 게임 시나리오로, 'Game'의 기본 골격('__init__', 'roll', 'score')
12   - 스페어/스트라이크가 없는 "정상 프레임"의 합산 로직을 가장 단순한 형태로 검증한다.
13   - 이후 사이클(올런 → 스페어 → 스트라이크 → 퍼펙트 → 10프레임)의 토대가 된다.
14
15 ## 다루지 않는 것 (Out of Scope)
16 - 0이 아닌 핀 수의 처리 (다음 사이클: "All Ones")
17 - 스페어/스트라이크 보너스 로직
18 - 10프레임 특수 규칙
19 - 입력 검증 (CLAUDE.md에 명시적으로 out of scope)
20 - 중간 프레임 점수 조회
21
22 ## 예상 실패 메시지
23 'game.py' 파일과 'Game' 클래스가 아직 존재하지 않으므로:
24
25 ...
26 ModuleNotFoundError: No module named 'game'
27 ...
28
29 (또는 'tests/test_game.py' 파일이 새로 작성되어야 하므로 임포트 단계에서 실패)
30
31 ## 파일 구조 (이 사이클에서 만든 것)
```

```
1 from game import Game
2
3
4 def test_gutter_game():
5     game = Game()
6     for _ in range(20):
7         game.roll(0)
8     assert game.score() == 0
9
```

GREEN 1 – 테스트를 통과시킬 최소 구현

Test Code 를 기반으로 빠르게 코드를 생성 후, Test Code 를 돌려본다.

```
CLAUDE.md  game.py x
1  class Game:
2      def __init__(self):
3          self._rolls = []
4
5      def roll(self, pins):
6          self._rolls.append(pins)
7
8      def score(self):
9          return sum(self._rolls)
10
```

```
• Bash(python -m pytest tests/ -v 2>&1)
  L ===== test session starts =====
    platform win32 -- Python 3.10.8, pytest-7.2.1, pluggy-1.0.0 --
    C:\Users\hwany\AppData\Local\Programs\Python\Python310\python.exe
    cachedir: .pytest_cache
    rootdir: C:\reviewer\workspace\bowling-tdd
    plugins: anyio-4.11.0
    collecting ... collected 1 item

    tests/test_game.py::test_gutter_game PASSED [100%]

    ===== 1 passed in 0.03s =====
```

REVIEW 1 – 생성된 코드 리뷰

코드를 검토한다.

Plan(Test Code) 에 맞도록 구현되었는가

- 지나치게 일반화 되거나, 불필요한 부분을 추가하지 않았는지
- 리팩토링이 필요한지

```
- 추가된 파일
- tests/test_game.py:1-8 - test_gutter_game 테스트 1개 추가. 20롤 모두 0이면 score() == 0을 검증.
- game.py:1-9 - Game 클래스 최소 구현 (__init__, roll, score).
- 목표 동작 충족 여부
- Plan.md의 목표("거터 게임은 0점")가 그대로 테스트로 표현되었고 통과함.
- 구현은 편 합산뿐 - 스페어/스트라이크 보너스는 의도적으로 미구현 (다음 사이클).
```

개선 후보 (실행은 승인 후)

```
- 현재 코드 9줄, 중복도 추상화 부담도 없음. 지금 단계에서 리팩터링할 만한 명확한 가치는 보이지 않음.
- (선택) tests/__init__.py를 명시적으로 추가할지 여부 - 현재 pytest가 자동 인식하므로 필요 없음.
```

잠재적 위험 / 미해결 항목

```
- 스페어 보너스(다음 1롤), 스트라이크 보너스(다음 2롤)는 아직 미구현 - sum()만으로는 절대 풀 수 없음. 다음 사이클부터 본격적인 점수
계산 로직 도입 필요.
- 10프레임 특수 규칙 미반영.
- 다음 사이클 후보 (CLAUDE.md의 권장 진행 순서):
  a. All Ones - 20롤 모두 1핀 → score() == 20 (sum())이 여전히 동작함을 확인하는 안전 단계)
  b. 그 다음 스페어 → 스트라이크 → 퍼펙트 → 10프레임
```

REVIEW 단계 자료를 준비했습니다. GREEN 단계의 코드와 개선 후보를 검토 부탁드립니다. 승인해 주시면 다음 사이클(All Ones)을 진행하겠습니다. 다른 우선순위(예: 바로 스페어로 점프)나 리팩터링 지시도 환영합니다.

RED 2 – Plan.md

다음 사이클에서도 Plan.md 를 받는다.

> 다음 사이클로 넘어가

- 다음 사이클 (All Ones)의 Plan.md를 작성하겠습니다.

```
M+ CLAUDE.md  game.py  M+ Plan.md x
1  # 사이클 2 - 올원 게임 (All Ones)
2
3  ## 목표 동작 (Behavior)
4  20번의 롤이 모두 1판일 때, `Game.score()`가 20을 반환한다.
5
6  ## 테스트 케이스
7  - 테스트 이름: `test_all_ones`
8  - 입력 / 셋업: 새로운 `Game()` 인스턴스 생성 후 `roll(1)`을 20회 호출
9  - 기대 결과 (assertion): `game.score() == 20`
10 - 왜 이 케이스가 가치 있는가?
11   - 0이 아닌 핀 수를 누적해서 합산하는 동작을 강제한다.
12   - 스페어/스트라이크가 발생하지 않는 "정상 프레임"의 합산을 거터 게임 외에 한 번 더 검증.
13   - 이후 스페어 사이클로 넘어갈 때 비교 기준점(보너스 미적용 vs 적용)을 제공한다.
14
15 ## 다루지 않는 것 (Out of Scope)
16 - 스페어 보너스 (다음 사이클)
17 - 스트라이크 보너스
18 - 10프레임 특수 규칙
19 - 입력 검증
20
21 ## 예상 실패 메시지
22 현재 `Game.score()`는 단순히 `sum(self._rolls)`를 반환하므로 - **이 테스트는 사실 통과할
23
```

- Plan.md 작성을 완료했습니다. 검토 부탁드립니다.

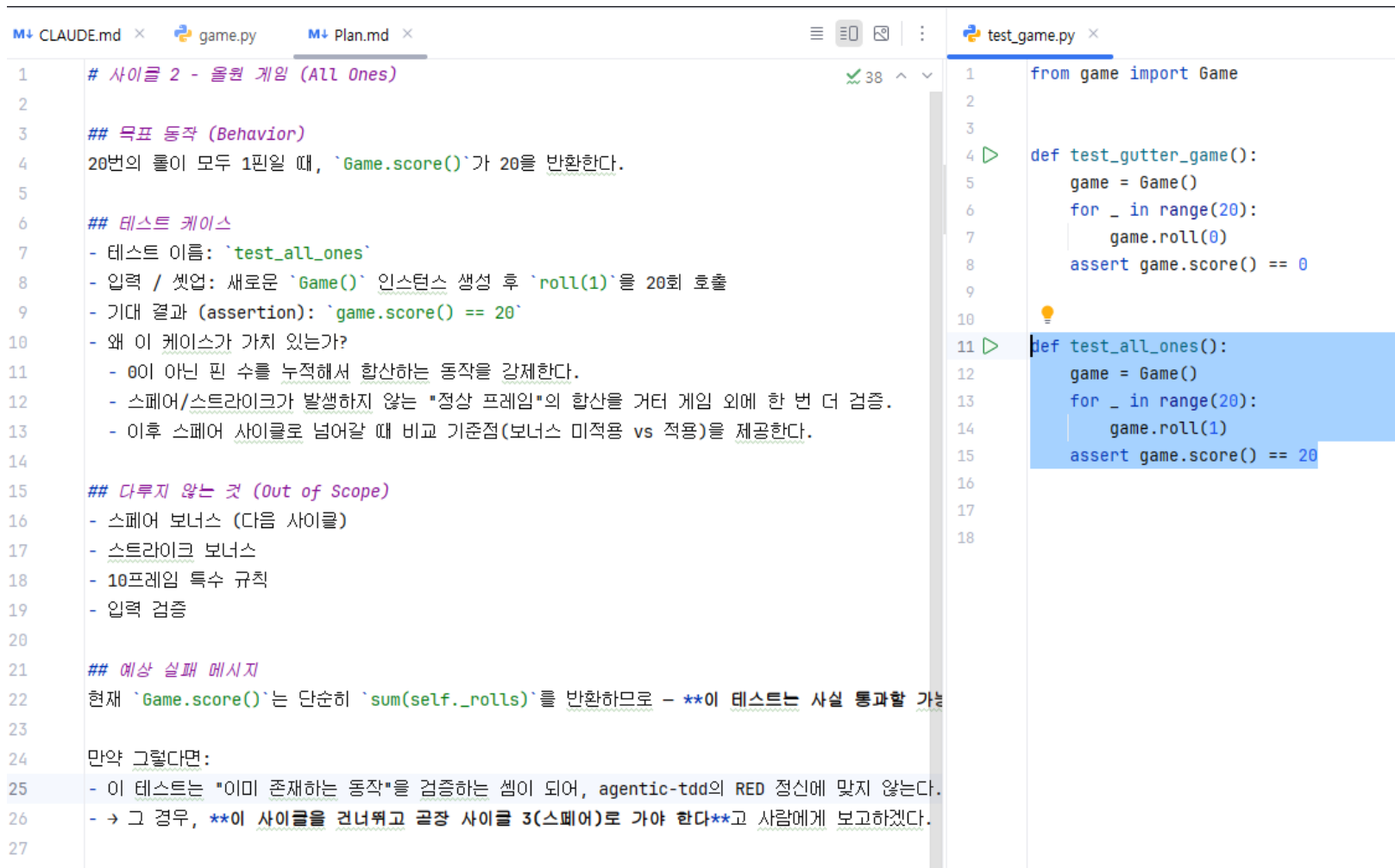
요약

- 사이클 2 목표: 올원 게임 - 20롤 모두 1이면 `score()`가 20
- 주의점: 현재 구현이 `sum()`이라 이 테스트는 즉시 통과할 가능성이 높음. 엄격한 RED를 못 만든다는 뜻.
- 결정 필요:
 - (A) 회귀 테스트로 유지 후 사이클 3(스페어)로 진행 - 추천
 - (B) 이 사이클 폐기, 바로 사이클 3(스페어)로 점프

(B)도 합리적입니다 - `agentic-tdd` 정신상 "이미 통과하는 테스트는 새 동작을 강제하지 못함"이기 때문입
알려주시면 그에 맞춰 RED 단계로 진행하겠습니다.

RED 2 – Test Code 변환 후 검토

마찬가지로 변환된 Test Code 가 Plan.md 의 목표에 맞도록 작성되었는지 확인한다.



The screenshot shows a code editor with two files open: Plan.md and test_game.py.

Plan.md (Left Panel):

```
1 # 사이클 2 - 올런 게임 (All Ones)
2
3 ## 목표 동작 (Behavior)
4 20번의 롤이 모두 1핀일 때, `Game.score()`가 20을 반환한다.
5
6 ## 테스트 케이스
7 - 테스트 이름: `test_all_ones`
8 - 입력 / 셋업: 새로운 `Game()` 인스턴스 생성 후 `roll(1)`을 20회 호출
9 - 기대 결과 (assertion): `game.score() == 20`
10 - 왜 이 케이스가 가치 있는가?
11   - 0이 아닌 핀 수를 누적해서 합산하는 동작을 강제한다.
12   - 스페어/스트라이크가 발생하지 않는 "정상 프레임"의 합산을 거터 게임 외에 한 번 더 검증.
13   - 이후 스페어 사이클로 넘어갈 때 비교 기준점(보너스 미적용 vs 적용)을 제공한다.
14
15 ## 다루지 않는 것 (Out of Scope)
16 - 스페어 보너스 (다음 사이클)
17 - 스트라이크 보너스
18 - 10프레임 특수 규칙
19 - 입력 검증
20
21 ## 예상 실패 메시지
22 현재 `Game.score()`는 단순히 `sum(self._rolls)`를 반환하므로 - **이 테스트는 사실 통과할 가능
23
24 만약 그렇다면:
25 - 이 테스트는 "이미 존재하는 동작"을 검증하는 셈이 되어, agentic-tdd의 RED 정신에 맞지 않는다.
26 - → 그 경우, **이 사이클을 건너뛰고 골장 사이클 3(스페어)로 가야 한다**고 사람에게 보고하겠다.
```

test_game.py (Right Panel):

```
1 from game import Game
2
3
4 def test_gutter_game():
5     game = Game()
6     for _ in range(20):
7         game.roll(0)
8     assert game.score() == 0
9
10
11 def test_all_ones():
12     game = Game()
13     for _ in range(20):
14         game.roll(1)
15     assert game.score() == 20
```

RED→REVIEW 2 – 테스트 검토

작성된 테스트코드가 GREEN 단계를 거치지 않고 통과되었다.
테스트가 유효한 테스트인지 확인후 넘어간다.

- 예상대로 RED 없이 즉시 통과했습니다. agentic-tdd의 정신상 진정한 RED 게이트를 통과한 것이 아닙니다 - `sum()`이 이미 이 동작을 충족하기 때문입니다.

전체 테스트도 확인합니다.

```
● Bash(python -m pytest tests/ -v 2>&1)
  L ===== test session starts =====
    platform win32 -- Python 3.10.8, pytest-7.2.1, pluggy-1.0.0 --
    C:\Users\hwany\AppData\Local\Programs\Python\Python310\python.exe
    cachedir: .pytest_cache
    rootdir: C:\reviewer\workspace\bowling-tdd
    plugins: anyio-4.11.0
    collecting ... collected 2 items

    tests/test_game.py::test_gutter_game PASSED [ 50%]
    tests/test_game.py::test_all_ones PASSED [100%]

    ===== 2 passed in 0.03s =====
```

RED 3 – Plan.md 를 테스트로

마찬가지로 Plan.md 를 검토 후 Test 생성을 한다.

```
M+ CLAUDE.md  game.py  M+ Plan.md x
1  # 사이클 3 - 스페어 보너스 (One Spare)
2
3  ## 목표 동작 (Behavior)
4  첫 프레임이 스페어(5+5=10)이고 그 다음 롤이 3이면, 첫 프레임 점수에 다음 1롤이 보너스로 더해져야 한다.
5  - 게임 시퀀스: `5, 5, 3, 0, 0, 0, ... (총 20롤)`
6  - 점수 계산: `프레임1 = 5+5+3(보너스) = 13`, `프레임2 = 3+0 = 3`, 나머지 = 0
7  - 합계: `13 + 3 = 16`
8
9  ## 테스트 케이스
10 - 테스트 이름: `test_one_spare`
11 - 입력 / 셋업:
12   - `Game()` 생성
13   - `roll(5)`, `roll(5)` (스페어)
14   - `roll(3)` (보너스 + 프레임2의 첫 롤)
15   - `roll(0)` 17회 (나머지 롤)
16 - 기대 결과 (assertion): `game.score() == 16`
17 - 왜 이 케이스가 가치 있는가?
18   - 단순 `sum()`으로는 절대 풀 수 없음 → **진정한 RED**가 보장됨.
19   - 스페어 = "두 롤의 합이 10"이라는 정의와 "다음 1롤을 보너스로 더한다"는 규칙을 동시에 강제.
20   - 다음 사이클(스트라이크)의 보너스 로직 일반화의 토대가 됨.
21
22 ## 다루지 않는 것 (Out of Scope)
23 - 스트라이크 (다음 사이클)
24 - 두 개 이상의 스페어 (Out of Scope - 한 케이스에 집중)
25 - 10프레임의 스페어/스트라이크 특수 처리
26 - 마지막 프레임이 스페어인 경우의 보너스
27 - 입력 검증
```

```
def test_one_spare():
    game = Game()
    game.roll(5)
    game.roll(5)
    game.roll(3)
    for _ in range(17):
        game.roll(0)
    assert game.score() == 16
```

GREEN 3 – 테스트 통과 확인

Test Code 가 잘 통과되는지 확인한다.

```
● Bash(python -m pytest tests/ -v 2>&1)
  | ===== test session starts =====
  | platform win32 -- Python 3.10.8, pytest-7.2.1, pluggy-1.0.0 --
  | C:\Users\hwany\AppData\Local\Programs\Python\Python310\python.exe
  | cachedir: .pytest_cache
  | rootdir: C:\reviewer\workspace\bowling-tdd
  | plugins: anyio-4.11.0
  | collecting ... collected 3 items
  |
  | tests/test_game.py::test_gutter_game PASSED [ 33%]
  | tests/test_game.py::test_all_ones PASSED [ 66%]
  | tests/test_game.py::test_one_spare PASSED [100%]
  |
  | ===== 3 passed in 0.02s =====
```

REVIEW 3

생성된 코드에 대해 리뷰한다.

```
class Game:
    def __init__(self):
        self._rolls = []

    def roll(self, pins):
        self._rolls.append(pins)

    def score(self):
        total = 0
        i = 0
        for _ in range(10):
            if self._rolls[i] + self._rolls[i + 1] == 10:
                total += 10 + self._rolls[i + 2]
            else:
                total += self._rolls[i] + self._rolls[i + 1]
            i += 2
        return total
```

REVIEW 3

필요시 리팩토링을 요청한다.

현재 생성된 코드 부분에서 `roll[i] + roll[i + 1] == 10` 을 검사하는 로직은 `is_spare` 와 같은 헬퍼함수를 도입해서 가독성을 높일 수 있을거 같아. 리팩토링 해줘



`</>` Edit automatically



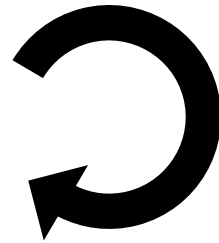
나머지 Cycle

나머지 Cycle 을 과제로 수행하도록 한다.

과제 수행 시 human-in-the-loop 가 잘 수행되었는지 체크용으로 Commit 을 만든다.



RED 단계에서 생성된
Plan.md Commit



GREEN 단계의 코드를 검토 후 Commit



Chapter 04

Agentic Engineering은 Over-engineering일까?

SW 품질을 위한 Agentic Engineering

아래 내용이 Agentic Engineering에 위배되는지 고민해보자. 1

- E2E 테스트 스크립트 적절히 추가해줘
- 이후 실행해보고 Fail되면 원인 분석 후 소스코드 or 스크립트를 개선해줘

아래 내용이 Agentic Engineering에 위배되는지 고민해보자. 2

위 방식은 완벽히 AE를 어기는 방식이다.

아래와 같이 해야 한다.

- E2E 테스트 스크립트 어떤 것을 만들면 좋을지 PLAN.md에 적어보라고 지시
 - 이후 인간의 검토 및 AI와 토론
- "테스트 스크립트 만들고 실행해" 지시
- (만약 Fail 발생시) 원인 분석과 수정 방향을 DEBUG.md에 작성해봐
 - 이후 인간의 검토 및 AI와 토론
- "수정 후 테스트까지 해봐" 지시

Test Code 만들 때에도 매번 PLAN 문서부터 만들라고 시켜야 할까?

TDD 할 때

- Code로 만들어진 것도 Plan 이라고 할 수 있지만, AI와 토론이 필요하다.
- Red 단계는 설계에 해당되며, 깊은 고민이 필요하다.
- Text로 만들어진 Plan을 작성, 검토 후에, Code로 만드는 것이 바람직하다.
- Text로 Plan 작성 → Code로 Plan 작성 (Unit Test)

가볍게 Test Case 1개 만들 때

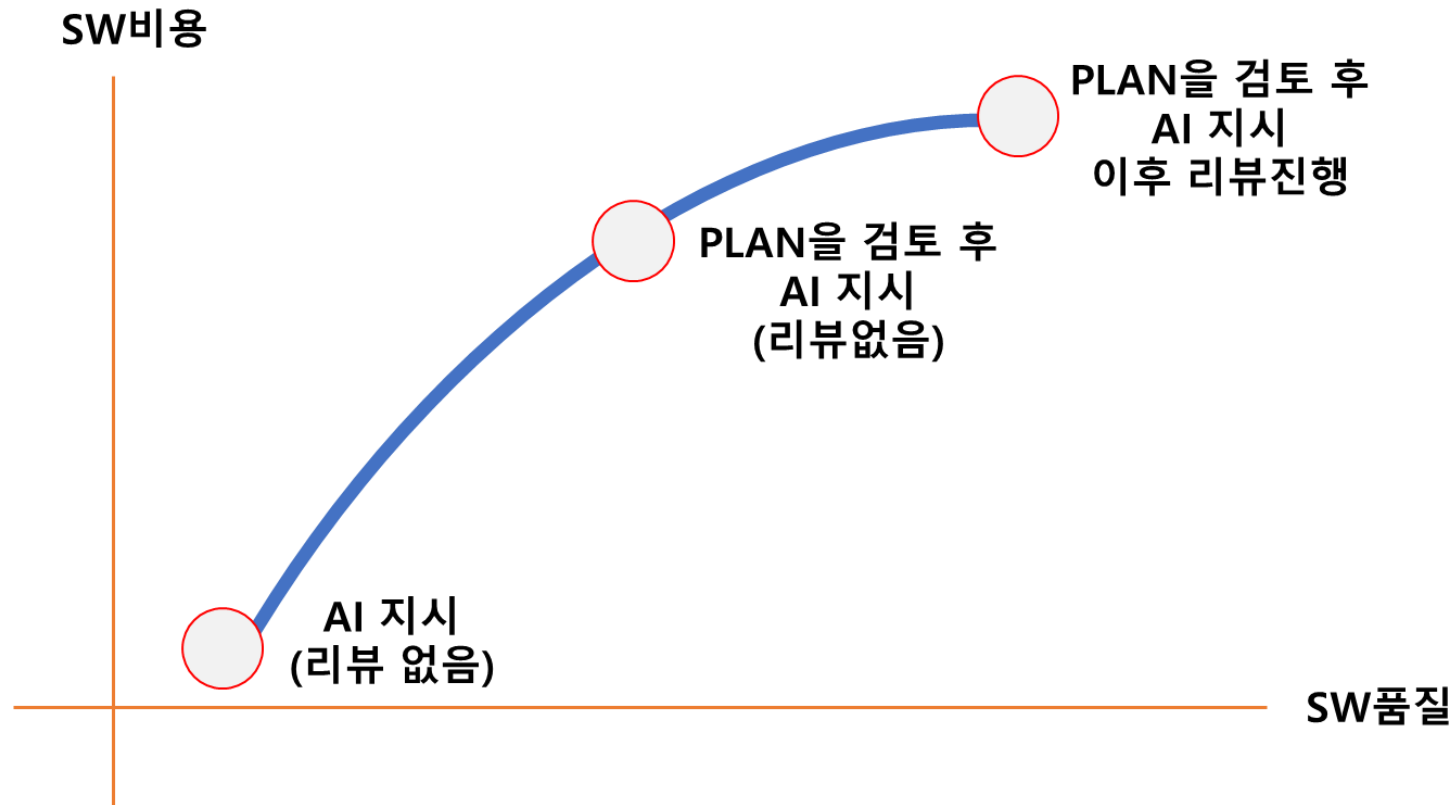
- [PLAN.md](#) 없이 대화로 어떤 것을 만들지 먼저 계획을 듣자. PLAN Mode만 사용해도 좋다.

Test Case 여러 개 만들 때

- [PLAN.md](#) 를 만들고, 사람 리뷰 & AI와 토론을 먼저 한다.

Agentic Engineering은 오버엔지니어링?

검토된 PLAN 없이 AI에게 작업을 시키고 리뷰를 하지 않는 것은 최악이라는 것은 동의할 수 있다.
AI에게 시킨 후 리뷰만 하면 되지 않나? 굳이 PLAN 문서로 대화 후, 시키고, 리뷰까지 해야 할까?



Agentic Engineering은 오버엔지니어링?

이유 1

- AI는 사람의 의도보다 더 많은 것을 구현한다. 따라서 AI가 무엇을 할지 미리 듣고 협의를 해야 한다.

이유 2

- AI에 맡기는 만큼 "요청문서 관리" 중요하다. "요청문서가 관리되지 않으면 안된다"

이유 3

- AI에게 시키고 후속 검증이 필요하다.

비유 - 억 단위 인테리어 어떻게 할까?

회사 입장에서는 SW 개발자가 만드는 제품은, SW 비용들이 포함된 고가의 제품이다.
이 제품을 만들 때는 '인테리어 할 때' 처럼 신중해야 한다.

인테리어 업자에게 전화로
"이쁘게 만들어 주세요"

VS

전체적으로 '이쁘게'에 대해
요구사항을 정리하고
각 단계별로 어떻게 만들 것인지 소통하고
한 단계 공사 후, 결과물에 대해 검토하고
수정 요청

MVP, PoC, 연구 수준이라면?

하네스 엔지니어링으로 충분하다.

Agentic 엔지니어링은 오버엔지니어링이다.

대신 실제 고객에게 제공되는 SW제품이라면?

- Agentic Engineering은 오버엔지니어링이 아니다. 필요한 엔지니어링이다.

Chapter 04

지속적인 발전을 위한 Agentic Engineering

SW 품질을 위한 Agentic Engineering

Agentic Engineering의 단점

- 똑딱 만드는 vibe coding에 비해, 개발 속도가 매우 느리다.
- Self 검토 및 AI와 토론, 팀 동료와 토론 및 공유 시간이 많이 들기 때문이다.
- 클린 코드 일수록 이 시간을 줄일 수 있지만, 여전히 리뷰 시간은 길다.

하지만 SW 품질을 위해 반드시 필요한 투자 시간이다.

SW품질 확보 외, Agentic Engineering의 장점

실력이 꾸준히 향상된다. AI로 인해 실력이 정체되는 것을 방지할 수 있다.

- 연차가 쌓일수록 SW 검토 실력이 지속적으로 늘어야 한다. 이건 중요한 문제이다.

반면, 바이트코딩에 과도하게 의존할 경우 코드 구현 역량이 점차 약화될 수 있다.

구현 경험이 줄어들수록 코드의 동작을 깊이 이해하는 능력과 검토, 판단 역량 역시 함께 약화될 가능성이 높다.

피할 수 없는 개발 인력의 축소, 어떻게 대응할까?

AI와 협업으로, 적은 인원으로도 SW 개발이 가능하다.

하네스 엔지니어링이 발전할수록 적은 실력으로도, OS급 SW개발도 가능해진다.

이런 발전 속도에서 SW 개발자의 경쟁력은 어디서 나올까?

- AI 없이도 개발할 수 있는 기본 SW 훈련 수준
 - SW 훈련이 잘 되어있을 수록 더 심도 있는 수준의 AI 토론이 가능하다.
- AI에게 사고를 의존하지 않고, AI 작업물에 대해 비판적 사고를 갖는다.
 - AI에게 냉철하게 질문한다.
- 알고리즘, 자료구조 능력 UP / 문제 분해 능력
 - AI가 만들 구현방식이 미리 예상이 되며, 이에 맞추어 Phase 분리를 잘 한다.
- AI가 만든 작업물을 검토하는 능력
 - 클린코드에 대한 고민을 많이 하여, 코드를 빠르게 이해하고 클린코드 방향성을 빠르게 캐치하는 능력

Agentic Engineering은 개발 역량을 지속적으로 올릴 수 있는 엔지니어링이다.

감사합니다.